

Audio Source Separation

Dalle Tecniche Tradizionali al Deep Learning

Andrea Bellu Brando Calderara

A.A. 2025/2026

Università degli Studi di Brescia

Definizione del Problema e Obiettivi

L'Audio Source Separation

- L'obiettivo è isolare specifici flussi sonori (sorgenti) a partire da un segnale misto (*canzone*).
- Si inquadra nel **Cocktail Party Problem**: la sfida di emulare la capacità dell'orecchio umano di discriminare un suono da altri.

Scopo del Progetto

- **Sperimentazione tradizionale**: implementazione della tecnica *Harmonic-Percussive Separation (HPS)*.
- **Deep Learning**: analisi e test del modello **Demucs**.
- **Valutazione**: confronto dei risultati e delle metriche di separazione.

Analisi in Frequenza: I Limiti

Discrete Fourier Transform (DFT):

- Analizza le frequenze presenti.
- **Problema:** perde il riferimento temporale.

Principio di Indeterminazione

Non è possibile conoscere con precisione assoluta e simultanea *quando* avviene un evento sonoro e *quale* frequenza lo compone.

Soluzione Necessaria:

- Serve una mappatura congiunta **Tempo-Frequenza**.

Short-Time Fourier Transform (STFT)

Come funziona:

- Framing
- Windowing
- Hop Size (H)

Lo Spettrogramma

Matrice derivata dalla STFT. Rappresenta visivamente l'energia del segnale:

- **Asse X:** Tempo
- **Asse Y:** Frequenza
- **Colore:** Intensità/Modulo

Metodi Statistici: Non-Negative Matrix Factorization (NMF)

Principio Matematico

Si scompone lo spettrogramma originale V nel prodotto di due matrici non negative:

$$V \approx W \cdot H$$

- **Dictionary Matrix (W):** contiene le basi spettrali o "firme" caratteristiche di ogni sorgente sonora.
- **Activation Matrix (H):** rappresenta l'involuppo temporale, indicando l'istante di attivazione di ogni componente di W .

Limiti dell'algoritmo

- **Instabilità:** l'algoritmo è soggetto al problema dei minimi locali.
- **Parametrizzazione:** il numero di sorgenti K viene definito preventivamente.
- **Sorgenti Astratte:** separazione matematica ma non *semantica*.

L'Approccio Classico: Harmonic-Percussive Separation

L'Idea di Base e la Pipeline

L'Idea di Base: Scomporre il segnale in due componenti: **armonica** e **percussiva**.

1. STFT → **2. Filtri Mediani** → **3. Maschere** → **4. iSTFT**

Maschere Tempo-Frequenza:

- **Binary:** Decisione netta (0 o 1).
- **Soft:** Rapporto probabilistico tra le energie.

Il ritorno al Tempo:

- Applicazione alla STFT originale ($\mathcal{X}$).
- **iSTFT** per ottenere l'audio finale.

Binary Masking (es. Armonica)

$$M_h(n, k) = \begin{cases} 1, & \text{se } \tilde{\mathcal{Y}}^h(n, k) \geq \tilde{\mathcal{Y}}^p(n, k) \\ 0, & \text{altrimenti} \end{cases}$$

Soft Masking (es. Armonica)

$$M_h(n, k) = \frac{\tilde{\mathcal{Y}}^h(n, k) + \frac{\epsilon}{2}}{\tilde{\mathcal{Y}}^h(n, k) + \tilde{\mathcal{Y}}^p(n, k) + \epsilon}$$

- **Fase Imprecisa:** il mascheramento agisce solo sul modulo. Riutilizzare la fase originale nella iSTFT genera gravi artefatti (*"suono metallico"*).
- **Semantica Assente:** l'algoritmo fatica a separare strumenti complessi.

L'avvento del Deep Learning: Demucs

Apprendimento automatico

- Il Deep Learning permette di apprendere le caratteristiche dei dati tramite strati (*layers*) gerarchici.
- L'unità fondamentale è il **neurone artificiale**.

Convolutional Neural Networks

- **Layer Convoluzionale:** estrazione di caratteristiche locali, *feature map*, tramite lo scorrimento di un *kernel*.
- **Layer di Attivazione:** introduzione della non linearità: **ReLU** ($f(x) = \max(0, x)$)
- **Layer di Pooling:** riduzione delle dimensioni spaziali (es. *Max Pooling*) e invarianza spaziale.

Il problema della profondità

Reti molto profonde rischiano la perdita di dettagli spaziali. Soluzione: **residual block**, in cui l'input originale aggira i layer convoluzionali e va a sommarsi direttamente al loro output.

Caratteristiche Fondamentali

- Progettata per produrre un output con **dimensioni identiche all'input**.
- Particolarmente adatta alla separazione audio. Mantiene la durata temporale.

Il modello **Demucs** adatta la potenza delle reti U-Net all'elaborazione sonora.

L'Innovazione: Dominio del Tempo

- Demucs opera direttamente sulla **waveform grezza** (forma d'onda nel dominio del tempo).
- Questo permette di preservare e impiegare le informazioni di fase per una ricostruzione più naturale e fedele.

- I segnali audio sono monodimensionali. Demucs utilizza **kernel 1D** (vettori di lunghezza K) che scorrono lungo l'asse del tempo per individuare pattern temporali e frequenziali.

Cosa cambia rispetto alle U-Net

- **Encoder:** il *max pooling* viene sostituito da convoluzioni 1D. La funzione ReLu viene sostituita con attivazioni **GLU (Gated Linear Unit)** che attuano una decisione dinamica.
- **BiLSTM (Bidirectional Long Short-Term Memory):** Fornisce al modello una "memoria" (layer BiLSTM), analizzando il brano in entrambe le direzioni per catturare dipendenze e contesto globale (ritmo e melodia).
- **Decoder e Output:** Tramite convoluzioni 1D trasposte e *skip connections*, in fase ascendente, la rete si dirama per generare simultaneamente 4 sorgenti: *Vocals, Bass, Drums, Other*.

Laboratorio Sperimentale

Obiettivo del Laboratorio

- Validazione e confronto tra l'approccio statistico tradizionale (**HPS**) e quello più moderno del Deep Learning (**Demucs**).
- Analisi della robustezza degli algoritmi su tracce audio reali in diverse condizioni di pulizia del segnale.

Dataset MUSDB18

- Utilizzo della libreria `musdb` per il caricamento di brani in formato non compresso, *wav*.
- Definizione del **Ground Truth**: sempre la stessa libreria fornisce gli stems originali isolati per il calcolo delle metriche di errore (implementato in `SpectrogramComparison.ipynb`).

L'ambiente di sviluppo e testing è stato interamente realizzato in Python, strutturando lo stack tecnologico nelle seguenti macro-aree:

- **Calcolo Scientifico:** `numpy` e `scipy` per la manipolazione matriciale e il filtraggio matematico.
- **Elaborazione Audio e Dataset:** `librosa` e `soundfile` per il calcolo delle trasformate (STFT/iSTFT) e l'I/O audio. Abbiamo usato la libreria `musdb` per accedere al dataset MUSDB18.
- **Deep Learning:** `demucs` per sfruttare il modello.
- **Visualizzazione:** `matplotlib` e `IPython` per il rendering degli spettrogrammi e il playback durante l'analisi.
- **Valutazione:** abbiamo usato la libreria `mir_eval` per la valutazione oggettiva della qualità delle tracce.

- **STFT**: Mappatura nel dominio tempo-frequenza con parametri fissi ($n_{fft} = 2048$, $hop = 512$).
- **Filtering**: Applicazione dei filtri mediani direzionali, finestra $L_h = 31$. Un filtro orizzontale isola le linee armoniche continue (Y_h), mentre un filtro verticale estrae gli impulsi percussivi (Y_p).
- **iSTFT**: Sintesi finale del segnale nel dominio del tempo.

Code Snippet

Estratti da HPS.ipynb

```
X = librosa.stft(x, n_fft=2048, hop_length=512)
Y = np.abs(X)**2

# Filtraggio ( $L_h = 31$ ,  $L_p = 31$ )
Y_h = median_filter(Y, size=(1, 31))
Y_p = median_filter(Y, size=(31, 1))

# Ricostruzione
x_h = librosa.istft(X_h)
```

Il Flusso di Lavoro:

- **1. Acquisizione:** conversione da matrice a tensore PyTorch. Controllo dimensionale e forzatura del formato stereo.
- **2. Inferenza:** passaggio all'API Separator. Il modello elabora la waveform ed estrae simultaneamente *vocals, bass, drums, other*.
- **3. Export:** i tensori di output vengono riconvertiti in array NumPy e salvati come `.wav`.

Core Snippet (Demucs.ipynb)

```
# 1. Preparazione del Tensore Stereo
wav_tensor = torch.tensor(audio).T.float()
if wav_tensor.ndim == 1:
    wav_tensor = wav_tensor.unsqueeze(0).repeat(2, 1)

# 2. Passaggio all'API
_, separated = separator.separate_tensor(
    wav_tensor, sr)

# 3. Post-processing ed Export
vocals = separated['vocals'].cpu().numpy()
sf.write("vocals.wav", vocals.T, sr)
```

Analisi visiva qualitativa Tempo-Frequenza

- Attraverso il confronto degli spettrogrammi, è possibile identificare visivamente l'efficacia della separazione operata dai due modelli.
- Si possono suddividere le imperfezioni nella separazione in: **Leakage (Sbavature)** e **Artifacts (Buchi)**.

Osservazioni sui risultati

- come è chiaro anche ad un primo ascolto, **HPS** tende a smembrare strumenti complessi, inserendo l'attacco percussivo in una traccia e la coda armonica nell'altra causa la mancanza di comprensione semantica (voce, chitarra...)
- grazie al bottleneck con memoria (BiLSTM), **Demucs** preserva meglio la struttura globale del brano e riduce i buchi spettrali.

Il Framework Matematico:

- Confronto imparziale tra le stime dei modelli e il **Ground Truth** (dataset MUSDB18).
- Metriche estratte: **SDR, SIR, SAR**.

Implementazione (mir_eval)

```
sdr, sir, sar, perm =  
mir_eval.separation.bss_eval_sources(  
    reference_sources, estimated_sources)
```

Risultati Sperimentali

Metrica	HPS	Demucs
SDR (dB)	<i>-4.05</i>	<i>7.05</i>
SIR (dB)	<i>1.35</i>	<i>14.98</i>
SAR (dB)	<i>-0.18</i>	<i>7.95</i>

Test di Robustezza: Addizione del Rumore

Stress Test:

- **Il Problema:** Le valutazioni su audio da studio non riflettono la realtà (es. registrazioni live sporche).
- **Metodo:** Iniezione di **Rumore Bianco Gaussiano** con un Rapporto Segnale-Rumore (SNR).

Le nostre ipotesi

HPS: essendo puramente geometrico, i filtri mediani verranno "accecati" dal rumore. Ci aspettiamo un risultato impreciso e "metallico".

Demucs: la rete, avendo appreso pattern semantici durante il training, dovrebbe agire da *denoiser* e non avere troppi problemi nella separazione.

Risultati Ottenuti

- **HPS:** il risultato pessimo di **SDR** conferma che l'algoritmo non è più in grado di distinguere l'energia del segnale da quella del rumore maschera.
- **Demucs:** mantiene prestazioni accettabili. La U-Net scarta la perturbazione ricostruendo le frequenze armoniche in modo coerente.

Risultati Sperimentali (traccia percussiva e batteria)

Metrica	HPS	Demucs
SDR (dB)	<i>-7.36</i>	<i>5.56</i>
SIR (dB)	<i>0.91</i>	<i>16.28</i>
SAR (dB)	<i>-4.08</i>	<i>6.05</i>

Dopo aver realizzato e utilizzato i due modelli siamo giunti alle seguenti conclusioni:

HPS

- **Pro:** leggero, veloce e deterministico (zero training).
- **Contro:** assenza di divisione semantica.
- **Sotto Stress:** i filtri geometrici non riescono a reggere il rumore.

Demucs

- **Pro:** alta qualità.
- **Contro:** architettura "pesante" e computazionalmente esosa.
- **Sotto Stress:** la rete agisce da *denoiser* garantendo una separazione accettabile delle tracce.

Grazie per l'attenzione

Progetto di Audio Source Separation